

LECTURE 6

DECLARING AND DEFINING FUNCTIONS

RECAP

- *datatype* name[*length_exp*];
- ARRAY NAME IS ADDRESS OF FIRST ELEMENT
- name[i] IS THE VALUE OF $(i + 1)^{th}$ ELEMENT
 - VALUE AT (ADDRESS OF BASE + i)

RECAP

- INITIALIZE USING LOOPS/LISTS
- MULTI-DIMENSIONAL ARRAYS
 - MATRICES, TENSORS, TABLES
- CHAR ARRAYS AS STRINGS
 - '\0' (NULL) IS APPENDED TO INDICATE EOS
 - `printf("%s",name);`
 - `scanf("%14s",name);`

RECAP

- SORTING ARRAYS
 - NATURAL SORT ($\approx$ BUBBLE SORT)
- SEARCHING ARRAYS
 - LINEAR SEARCH
 - BINARY SEARCH FOR SORTED

RE-LOOK AT STANDARD LIBRARY FUNCTIONS

RE-LOOK: HELLO WORLD!

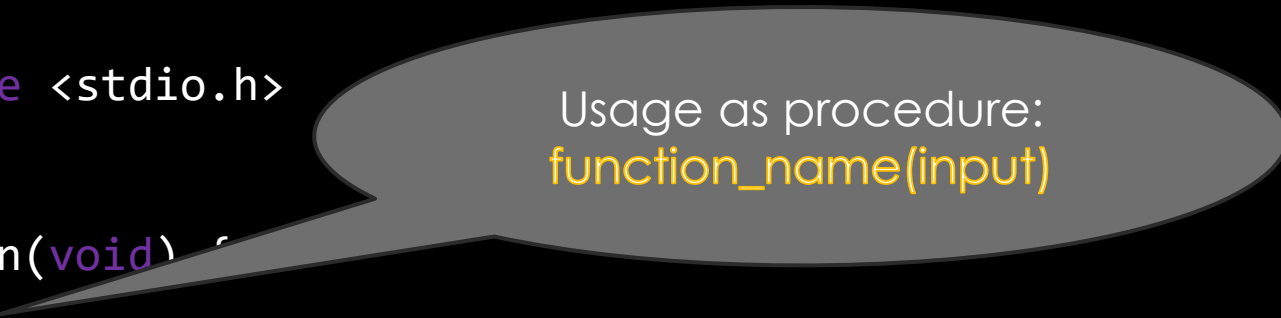
```
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

RE-LOOK: HELLO WORLD!

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello World!\n");  
    return 0;  
}
```



Usage as procedure:
`function_name(input)`

RE-LOOK: HELLO WORLD!

```
#include <stdio.h>
```



Declared
here

```
int main(void) {  
    printf("Hello World!\n");  
    return 0;  
}
```


STDIO.H

```
#ifndef _EXFUN
```

```
    # define _EXFUN(N,P) N P    %    _EXFUN(N,P) as a macro that simply expands to N P  
#endif
```

```
int _EXFUN(sprintf, (const char *, ...));
```

simply declares the prototype for printf

```
_EXFUN(sprintf, (const char *, ...))
```

expands to:

```
c
```

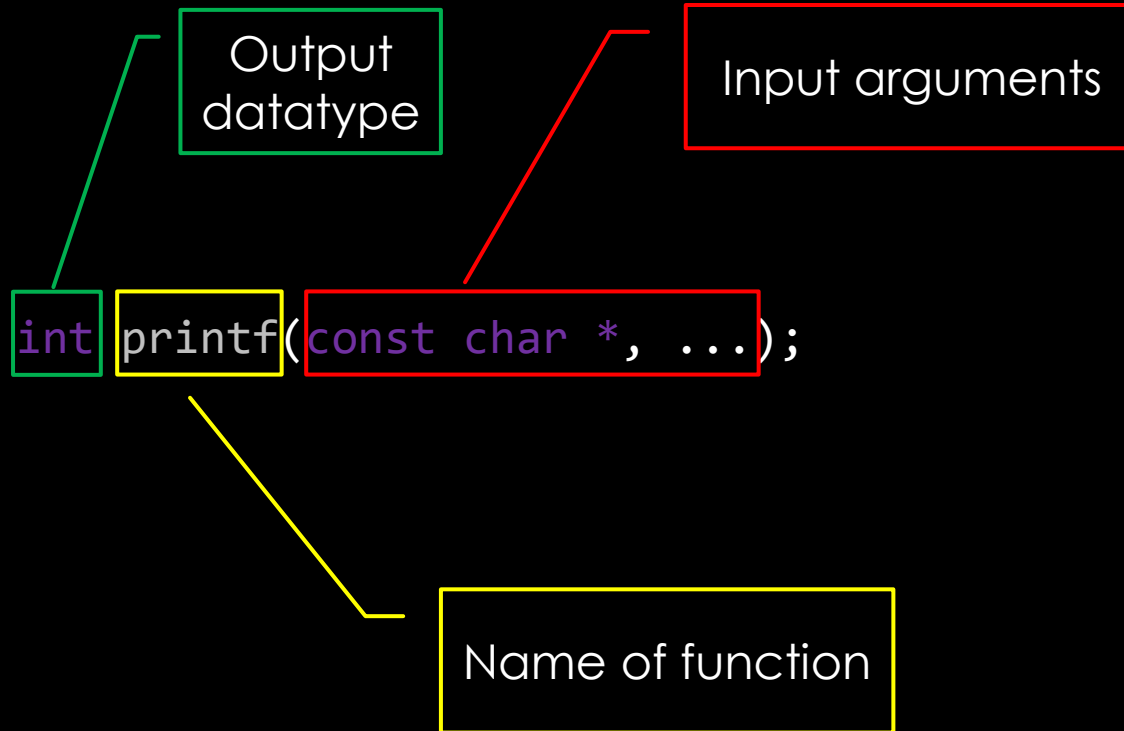
```
sprintf (const char *, ...)
```

STDIO.H

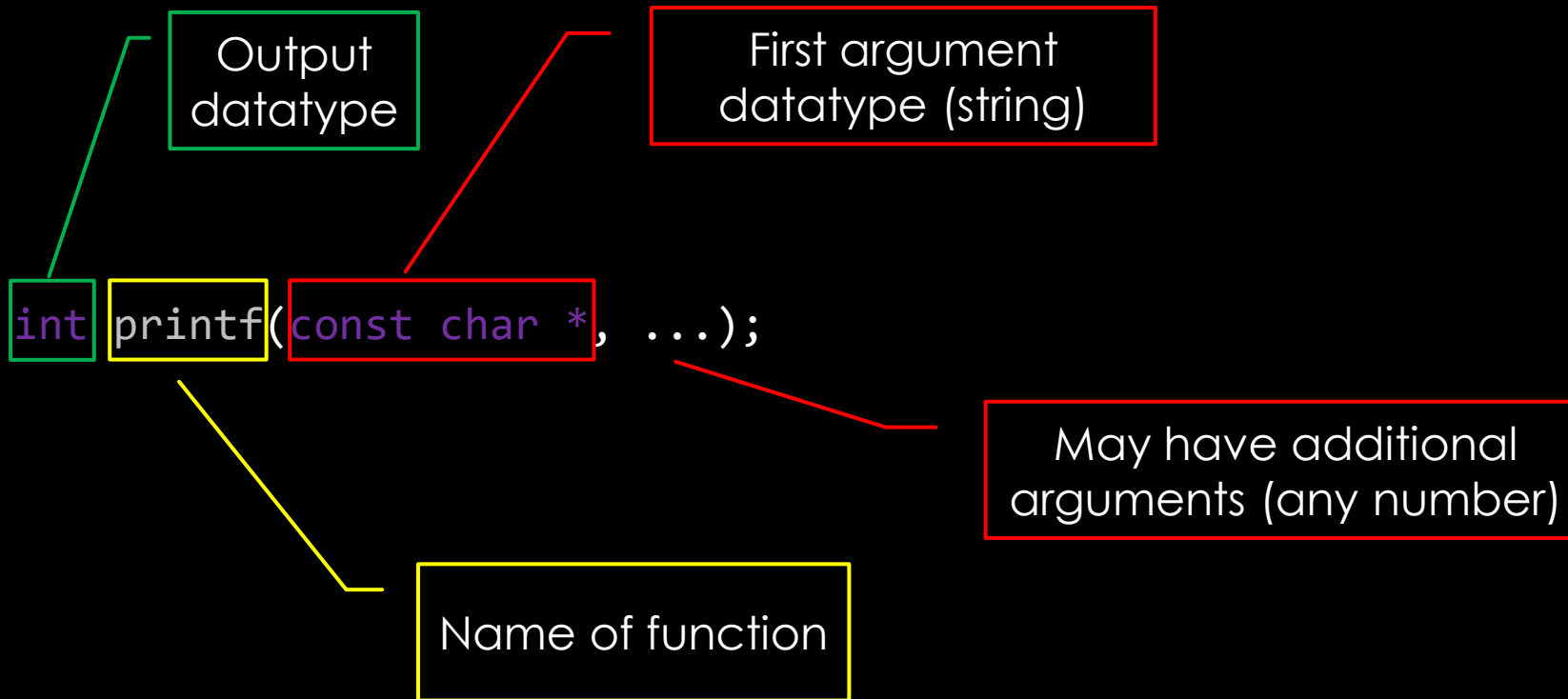
```
int printf(const char *, ...);
```

DECLARING FUNCTIONS

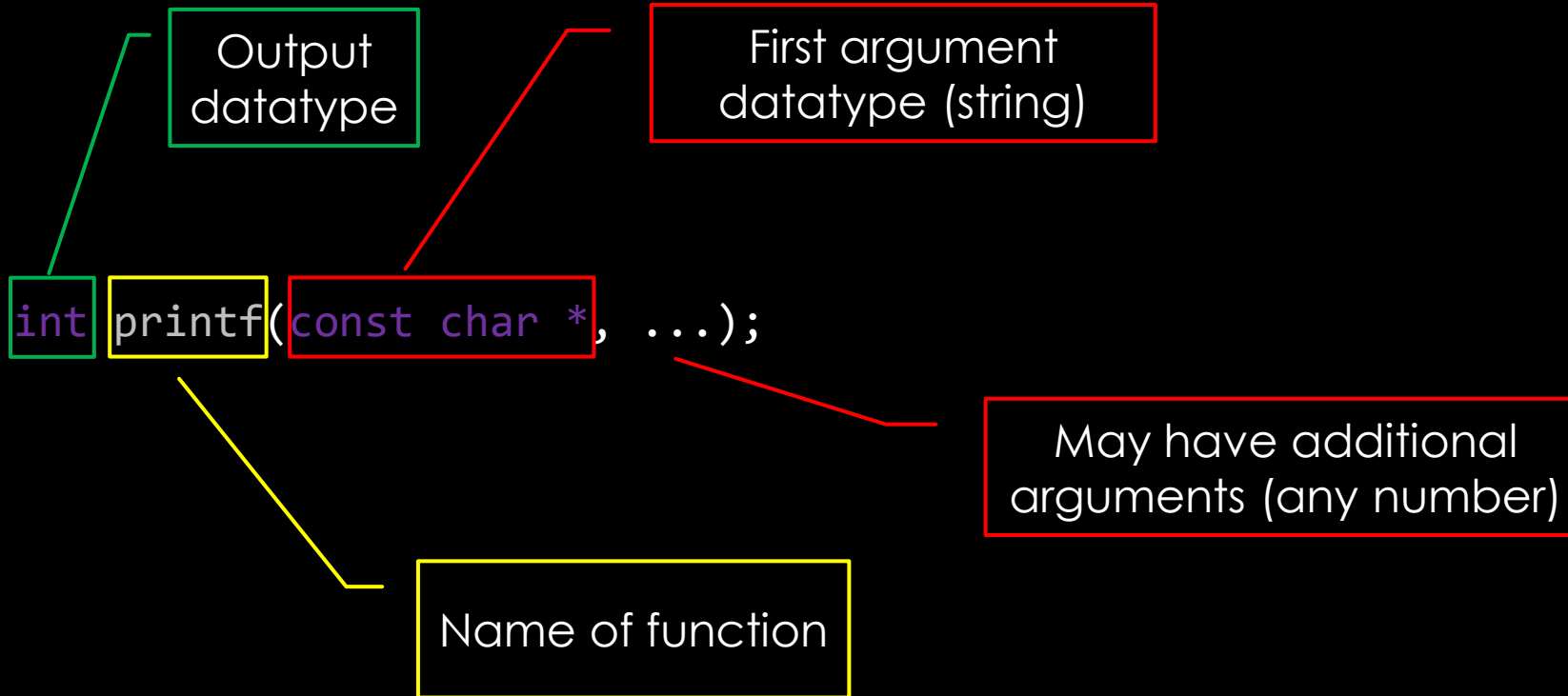
DECLARING FUNCTIONS (PROTOTYPE)



DECLARING FUNCTIONS



DECLARING FUNCTIONS (VARIADIC)



WHERE IS THE CODE FOR `printf` ?

- DECLARING A FUNCTION IS ENOUGH BEFORE CALLING IT IN YOUR CODE
- DEFINITION (CODE) CAN BE ANYWHERE OUTSIDE MAIN
 - STANDARD LIBRARY DEFINITIONS
 - ARE IN SEPARATE FILES (`PRINTF.C`)
 - PRE-COMPILED (`PRINTF.O`)
 - LINKED BY COMPILER AUTOMATICALLY

VOLUME OF SPHERE !

The volume of a d -dimensional sphere (radius r) is:

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2} + 1\right)} r^d$$

where

- $\Gamma(\cdot)$ is the Gamma function, which generalizes factorial: $\Gamma(n + 1) = n!$.
- This formula reduces to familiar cases:
 - $d = 2$: $V_2(r) = \pi r^2$ (area of a circle).
 - $d = 3$: $V_3(r) = \frac{4}{3}\pi r^3$ (volume of a sphere).

VOLUME OF SPHERE !

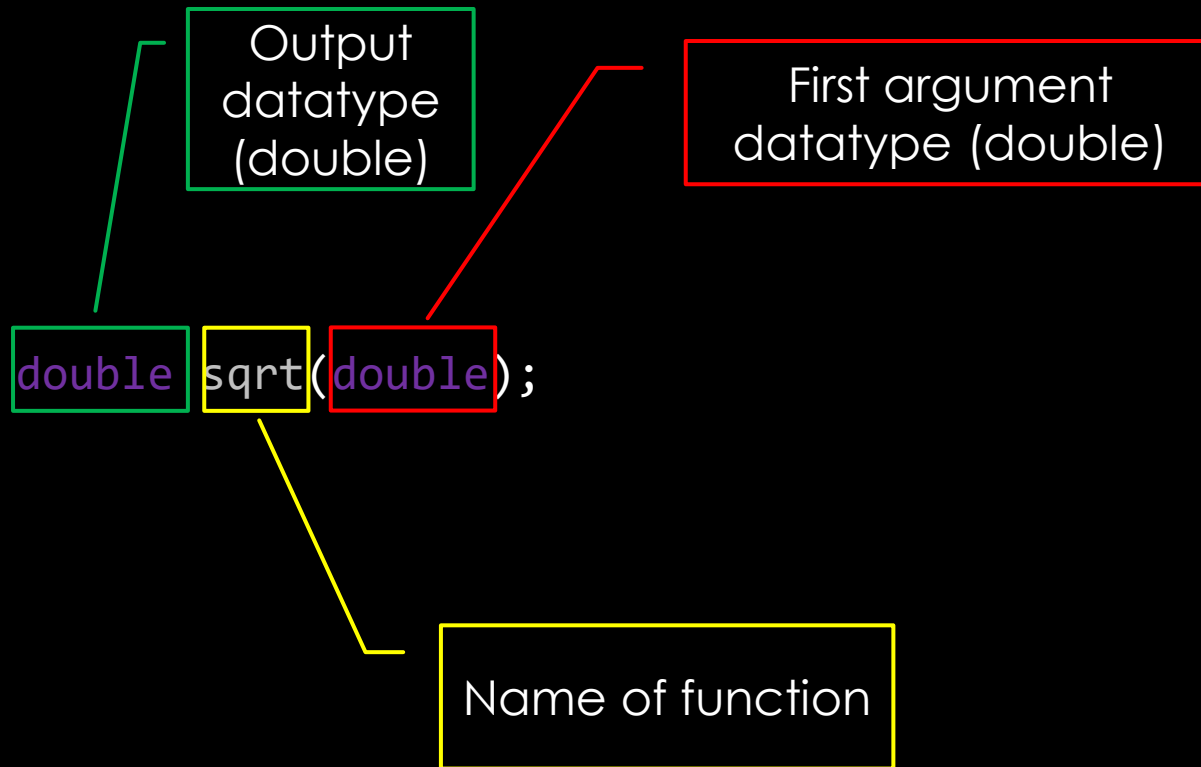
```
#include <stdio.h>
```

```
#include <math.h>
```

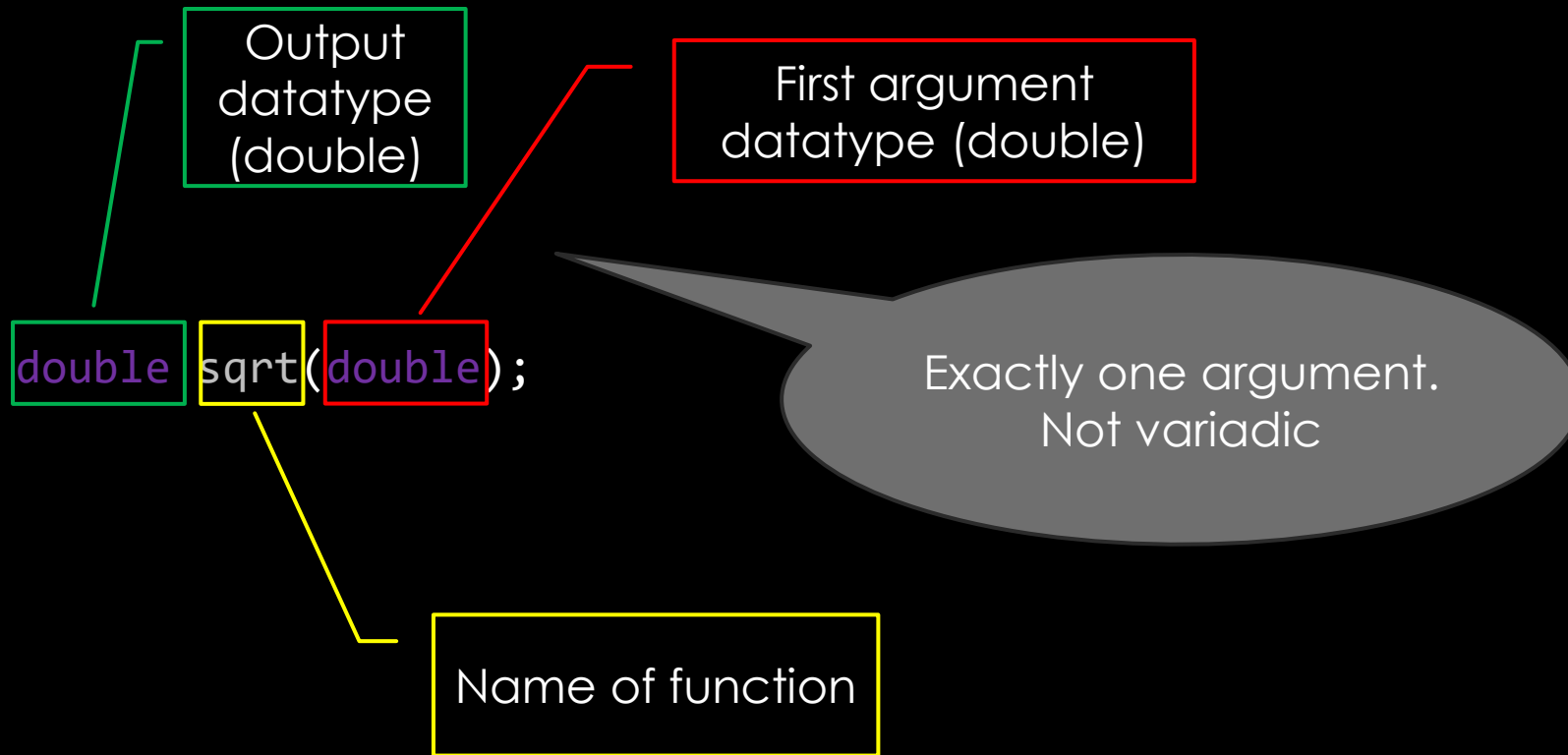
```
int main(void) {  
    double radius, volume;  
    int dimension;  
    printf("Enter radius,dimension for the hypersphere:\n");  
    scanf("%lf,%d",&radius, &dimension);  
    volume = pow(sqrt(M_PI)*radius,dimension)/exp(lgamma(dimension/2.0+1));  
    printf("Volume=%lf\n",volume);  
    return 0;  
}
```

Usage as function:
output➤function_name(input)

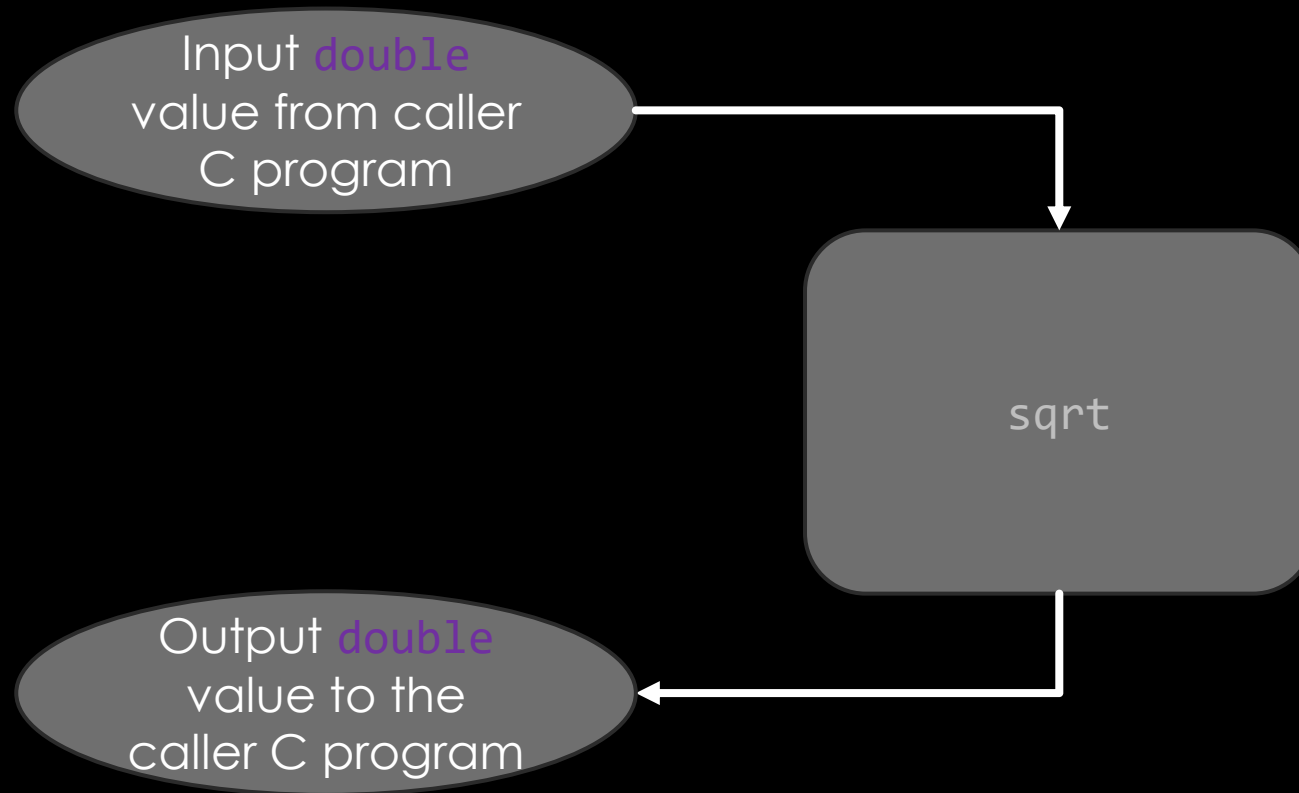
DECLARATION OF `sqrt` IN MATH LIBRARY



DECLARATION OF `sqrt` IN MATH LIBRARY



VISUALIZING sqrt



DEFINING A NEW FUNCTION

DEFINING A NEW FUNCTION: SIMPLE EXAMPLE

```
int square(int); // Declaration of my custom function
```

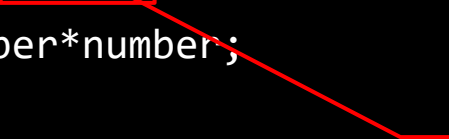
```
int square(int number){  
    return number*number;  
}
```

DEFINING A NEW FUNCTION: SIMPLE EXAMPLE

```
int square(int); // Declaration of my custom function
```

```
/*Below is the definition of square function*/
```

```
int square(int number){  
    return number*number;  
}
```



Stores input value from caller C program in new local variable number

DEFINING A NEW FUNCTION: SIMPLE EXAMPLE

```
int square(int); // Declaration of my custom function
```

```
/*Below is the definition of square function*/
```

```
int square(int number){  
    return number*number;  
}
```



Let's store this definition
in square.c file

USING A NEW FUNCTION: SIMPLE EXAMPLE

```
#include <stdio.h>

int square(int); // Declare custom function in YOUR program before using it

int main(void) {
    int my_number; printf("Enter a number:"); scanf("%d",&my_number);
    printf("Square of this number is:%d\n",square(my_number));
    return 0;
}
```

USING A NEW FUNCTION: SIMPLE EXAMPLE

```
#include <stdio.h>

int square(int); // Declare custom function in YOUR program before using it

int main(void) {
    int my_number; printf("Enter a number:"); scanf("%d",&my_number);
    printf("Square of this number is:%d\n",square(my_number));
    return 0;
}
```



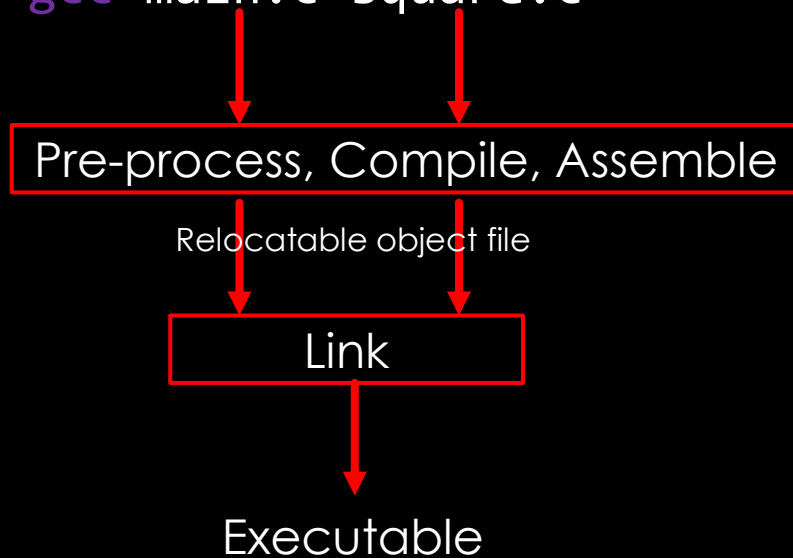
Let's store this definition
in main.c file

COMPILING MULTIPLE FILES

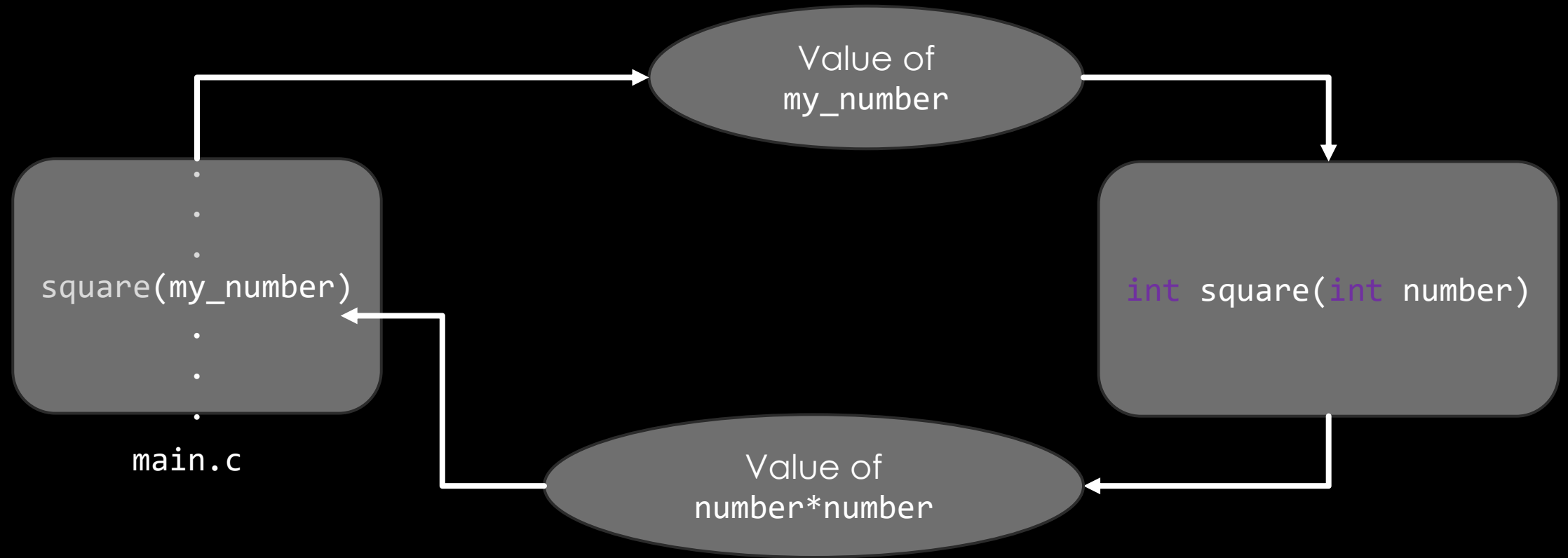
```
bash$ gcc main.c square.c
```

COMPILING MULTIPLE FILES

```
bash$ gcc main.c square.c
```



VISUALIZING a.out



DISTRIBUTION OF CODE



Let's include this
definition in id1063.h file

```
int square(int); // Declaration of my custom function
```

```
/*Below is the definition of square function*/
```

```
int square(int number){  
    return number*number;  
}
```



Let's store this definition
in square.c file

USING A NEW FUNCTION: SIMPLE EXAMPLE

```
#include <stdio.h>
#include <id1063.h>    //Contains declaration of square

int main(void) {
    int my_number; printf("Enter a number:"); scanf("%d",&my_number);
    printf("Square of this number is:%d\n",square(my_number));
    return 0;
}
```

DISTRIBUTION OF CODE

```
bash$ gcc -c square.c
```



Pre-process, Compile, Assemble



Relocatable object file
square.o

DISTRIBUTION OF CODE

1. DISTRIBUTE HEADER FILE id1063.h AND PRE-COMPILED square.o
2. STORE IN STANDARD PATH

```
bash$ gcc main.c
```



Pre-process, Compile, Assemble, Link



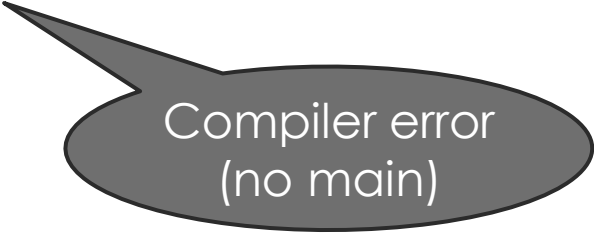
Executable
a.out

WHAT HAPPENS?

```
bash$ gcc square.c
```

WHAT HAPPENS?

```
bash$ gcc square.c
```



Compiler error
(no main)

```
gcc -c square.c  
gcc -c main.c  
gcc main.o square.o -o app
```

C FUNCTIONS

- CODE REUSE
- HELP MODULARIZE CODE
 - IMPROVES READABILITY
 - EASY TO DEBUG
- REPRESENT MATHEMATICAL/LOGICAL FUNCTIONS

MODULAR PROGRAM

```
#include <stdio.h>

int square(int);

int main(void) {
    int my_number; printf("Enter a number:"); scanf("%d",&my_number);
    printf("Square of this number is:%d\n",square(my_number));
    return 0;
}

int square(int number){
    return number*number;
}
```

COMPOSITION EXAMPLE

```
#include <stdio.h>

int square(int);

int main(void) {
    int my_number; printf("Enter a number:"); scanf("%d",&my_number);
    printf("Square of this number is:%d\n", square(square(my_number)));
    return 0;
}

int square(int number){
    return number*number;
}
```

FACTORIAL: REVISIT

```
#include <stdio.h>
#include <math.h>
int main(void){
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
    double factorial=0;
    for(unsigned int i=2;i<=number;i++){
        factorial += log(i);
    }
    printf("%u!=%e.\n",number,exp(factorial));
    return 0;
}
```

FACTORIAL: REVISIT

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main(void){
```

```
    unsigned int number; printf("Enter number:\n"); scanf("%u",&number);
```

```
    double factorial=0;
```

```
    for(unsigned int i=2;i<=number;i++){
```

```
        factorial += log(i);
```

```
    }
```

```
    printf("%u!=%e.\n",number,exp(factorial));
```

```
    return 0;
```

```
}
```


factorial.c

```
#include <math.h>

double factorial(unsigned int number){
    double logfactorial=0;
    for(unsigned int i=2;i<=number;i++){
        logfactorial += log(i);
    }
    return exp(logfactorial);
}
```

FACTORIAL: MODULAR PROGRAM

```
#include <stdio.h>

#include <id1063.h> //Contains declaration double factorial(unsigned int)

int main(void){
    unsigned int my_number; printf("Enter number:\n"); scanf("%u",&my_number);
    printf("%u!=%e.\n",number,factorial(my_number));
    return 0;
}
```

COMPOSITION EXAMPLE

```
#include <stdio.h>

#include <id1063.h> //Contains declaration double factorial(unsigned int)

int main(void){
    unsigned int my_number; printf("Enter number:\n"); scanf("%u",&my_number);
    printf("%u!=%e.\n",number,sqrt(factorial(my_number)));
    return 0;
}
```

IS SAME NAME OK?

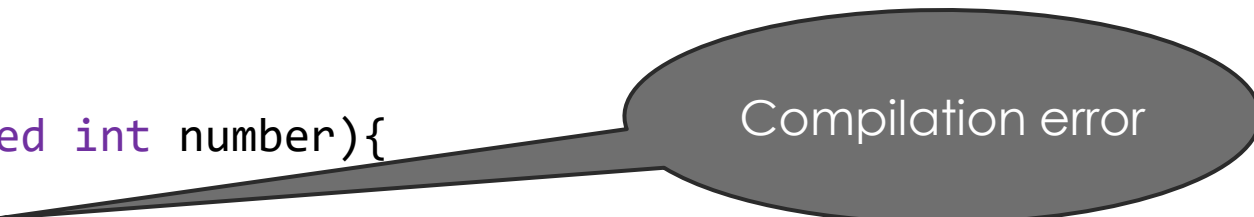
```
#include <math.h>

double factorial(unsigned int number){
    double factorial=0;
    for(unsigned int i=2;i<=number;i++){
        factorial += log(i);
    }
    return exp(factorial);
}
```

IS SAME NAME OK?

```
#include <math.h>
```

```
double factorial(unsigned int number){  
    double factorial=0;  
    for(unsigned int i=2;i<=number;i++){  
        factorial += log(i);  
    }  
    return exp(factorial);  
}
```



Compilation error

REVISITING PRINT A DIAMOND SHAPE

[illegible]

REVISITING PRINT A DIAMOND SHAPE

- LET R DENOTE THE NUMBER OF ROWS (ODD)
- NO. STARS IN r^{th} ROW: $2 \min(r, R - r + 1) - 1$

- SPACES BEFORE STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$

- SPACES AFTER STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$

REVISITING PRINT A DIAMOND SHAPE

- LET R DENOTE THE NUMBER OF ROWS (ODD)
- NO. STARS IN r^{th} ROW: $2 \min(r, R - r + 1) - 1$
 - SPACES BEFORE STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$
 - SPACES AFTER STARS: $\frac{(R - (2 \min(r, R - r + 1) - 1))}{2}$

REVISITING PRINT A DIAMOND SHAPE

```
#include <stdio.h>

int main(void){
    unsigned int totalRows; printf("Enter an odd number:\n"); scanf("%u",&totalRows);
    unsigned int no_spaces, no_stars;
    for(unsigned int r=1;r<=totalRows;r++){
        no_stars=2*((r<=(totalRows-r+1)) ? r : (totalRows-r+1))-1;
        no_spaces=(totalRows-no_stars)/2;
        for(unsigned int i=1;i<=no_spaces;i++) printf(" ");
        for(unsigned int i=1;i<=no_stars;i++) printf("*");
        for(unsigned int i=1;i<=no_spaces;i++) printf(" "); printf("\n");
    }
    return 0;}
```

print_diamond FUNCTION

```
#include <stdio.h>

void print_diamond(unsigned int totalRows){
    unsigned int no_spaces, no_stars;
    for(unsigned int r=1;r<=totalRows;r++){
        no_stars=2*((r<=(totalRows-r+1)) ? r : (totalRows-r+1))-1;
        no_spaces=(totalRows-no_stars)/2;
        for(unsigned int i=1;i<=no_spaces;i++) printf(" ");
        for(unsigned int i=1;i<=no_stars;i++) printf("*");
        for(unsigned int i=1;i<=no_spaces;i++) printf(" "); printf("\n");
    }
    return;    //Can be skipped as no statements follow
}
```

main.c PROGRAM

```
#include <stdio.h>

#include <id1063.h> //Declaration of print_diamond

int main(void){
    unsigned int totalRows; printf("Enter an odd number:\n"); scanf("%u",&totalRows);
    print_diamond(totalRows);
    return 0;
}
```

REPEATED PRINTING

```
#include <stdio.h>

#include <id1063.h> //Declaration of print_diamond

int main(void){
    unsigned int totalRows; printf("Enter an odd number:\n"); scanf("%u",&totalRows);
    for(short int i=1;i<=totalRows;i+=2)
        print_diamond(i);
    return 0;
}
```

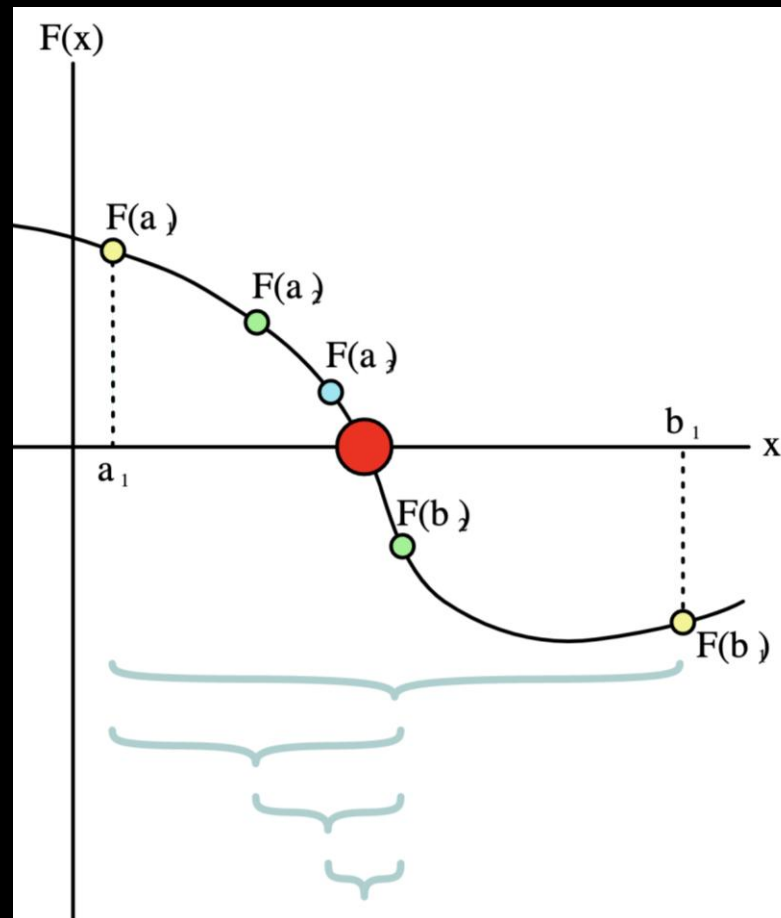
FINDING ROOTS OF AN IMPLICIT FUNCTION

- LET FUNCTION EVALUATION BE DEFINED IN `foo.c` (IMPLICIT FUNCTION)
 - `double foo(double);`
- THINK OF FUNCTION AS A DATABASE. CAN WE RUN BINARY SEARCH ?

FINDING ROOTS OF AN IMPLICIT FUNCTION

- LET FUNCTION EVALUATION BE DEFINED IN `foo.c` (IMPLICIT FUNCTION)
 - `double foo(double);`
- THINK OF FUNCTION EVALUATIONS AS A DATABASE. CAN WE RUN BINARY SEARCH ?

FINDING ROOTS OF AN IMPLICIT FUNCTION



BINARY SEARCH

```
#include <stdio.h>

double foo(double);

int main(void){
    double left,right,middle; printf("Enter interval containing a root [a,b]:");
    scanf("[%lf,%lf]",&left,&right);
    while(right-left > 1e-10){
        middle=(left+right)/2;
        if(foo(middle)==0){break;}
        else if(foo(middle)*foo(left)<=0) right=middle;
        else left=middle;}
    printf("Root at %lf: f(%lf)=%lf\n",middle,middle,foo(middle));
    return 0;
}
```


BINARY SEARCH

```
double foo(double x){  
    return x*x*x-x-2; // Or any other function over reals or even implicit function  
}
```

WHAT'S THE OUTPUT?

```
double foo(double x){  
    return x*x-2;  
}
```

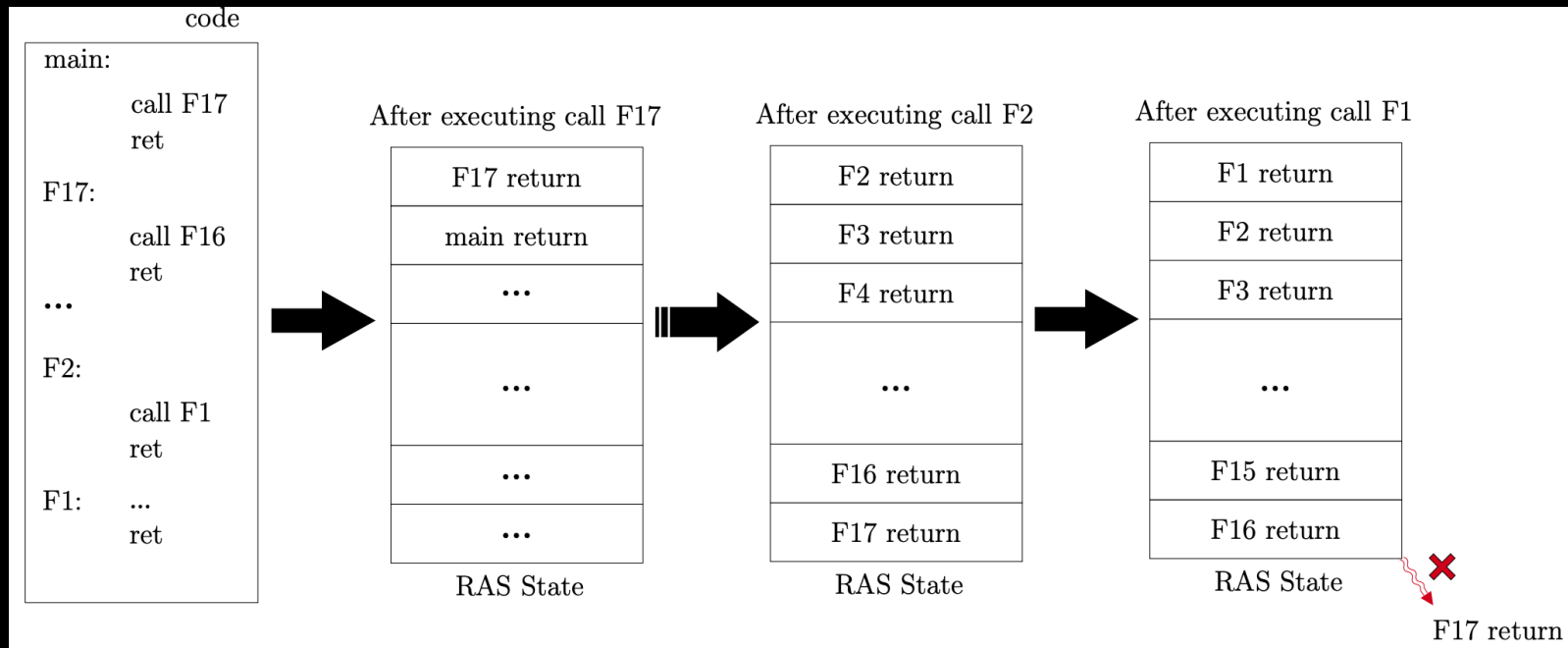
WHAT'S THE OUTPUT?

```
double foo(double x){  
    return x*x-2; //approximation of sqrt(2)  
}
```

BEHIND THE SCENES: STACK FRAMES

- DATA STRUCTURE USED BY C TO FACILITATE NESTED FUNCTION CALLING
- THINK OF PILE OF DISHES
- EACH STACK FRAME (DISH) HAS RETURN ADDRESS STORED
- EACH CALL "PUSHES" A STACK FRAME INTO THE STACK (PILE)
- EACH RETURN "POPS" A STACK FRAME FROM THE STACK

WORKING OF FUNCTION-CALL STACK



STACK FRAMES

- CONTAIN THE RETURN ADDRESS
- VARIABLES DECLARED INSIDE FUNCTION (DYNAMICALLY ALLOCATED AT RUN-TIME)